

| DS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Session | Queue | Question Information | Level 1 | Challenge 55 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------|----------------------|---------|--------------|
| <p><b>Question description</b></p> <p>Sathya is an DS expert training youngsters struggling in DS to make them better.</p> <p>Sathya usually gives interesting problems to the youngsters to make them love the DS.</p> <p>One such day Sathya provided to the youngsters to solve the task such that, insert an element in a Queue in FIFO order</p> <p>Youngsters were lacking the idea to solve the problem.</p> <p>Being an exciting youngster can you solve it?</p> <p><b>Function Description</b></p> |         |       |                      |         |              |
| <p>CH.SC.U4CSE25225 - M ASHWIN   Department of Computer Science &amp; Engineering (CSE)   2026</p>                                                                                                                                                                                                                                                                                                                                                                                                          |         |       |                      |         |              |

1. Define the maximum size of queue and initialize front and rear as -1.
2. In the main function we will initialize two variables that will store the data and the size of the queue.
3. Accept the data that we want to enter in a queue using a for loop.
4. After accepting the data use enqueue() function to insert the data in a queue.
5. In this function return queue id full , if the value of rear is equal to max-1.
6. Else increase the value of rear by 1.
7. After this insert the data that we have accepted earlier.
8. Now display the data of the queue using a disp() function.
9. In this function if the value of front is equal to -1 then it means list is empty.
10. Else initialize a for loop from front to rear and print the data that queue is holding.

**Constraints:**

$0 < \text{size} < 100$

$0 < \text{data} < 1000$

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5  
5 6 3 1 2

EXPECTED OUTPUT

Enqueueing 5  
5 Enqueueing 6  
5 6 Enqueueing 3  
5 6 3 Enqueueing 1  
5 6 3 1 Enqueueing 2  
5 6 3 1 2

Test Case 2

INPUT (STDIN)

8  
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

Enqueueing 1  
1 Enqueueing 8  
1 8 Enqueueing 6  
1 8 6 Enqueueing 4  
1 8 6 4 Enqueueing 9  
1 8 6 4 9 Enqueueing 5  
1 8 6 4 9 5 Enqueueing 3  
1 8 6 4 9 5 3 Enqueueing 5  
1 8 6 4 9 5 3 5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int data)

Test Case 2

KEYWORD

for(i=front;i<=rear;i++)

Test Case 3

KEYWORD

enqueue(data);

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

| CYCLOMATIC COMPLEXITY | TOKEN COUNT | NLOC |
|-----------------------|-------------|------|
| 3                     | 200         | 44   |

```
Queue5.c +
1 #include <stdio.h>
2
3 #define MAX 100
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int data) {
10     if (rear == MAX - 1) return;
11     if (front == -1) front = 0;
12     queue[++rear] = data;
13 }
14
15 void disp() {
16     int i;
17     for (i = front; i <= rear; i++) {
18         printf("%d ", queue[i]);
19     }
20 }
21
22 int main() {
23     int size;
24     if (scanf("%d", &size) != 1) return 0;
25
```

```
26  for (int i = 0; i < size; i++) {  
27      int data;  
28      scanf("%d", &data);  
29      if (i > 0) {  
30          disp();  
31      }  
32      printf("Enqueuing %d\n", data);
```

```
33     enqueue(data);
34 }
35 disp();
36 printf("\n");
37 return 0;
38}
39
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

5 6 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue5.exe

Enqueuing 5

5 Enqueuing 6

5 6 Enqueuing 3

5 6 3 Enqueuing 1

5 6 3 1 Enqueuing 2

5 6 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue5.exe

Enqueuing 1

1 Enqueuing 8

1 8 Enqueuing 6

1 8 6 Enqueuing 4

1 8 6 4 Enqueuing 9

1 8 6 4 9 Enqueuing 5

1 8 6 4 9 5 Enqueuing 3

1 8 6 4 9 5 3 Enqueuing 5

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>